VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

AISHWARYA M (1BM24CS024)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by AISHWARYA M(1BM24CS024), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026 . The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Faculty Incharge Name
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program 1a)

Question: Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

→FCFS

→ SJF (pre-emptive & Non-preemptive)

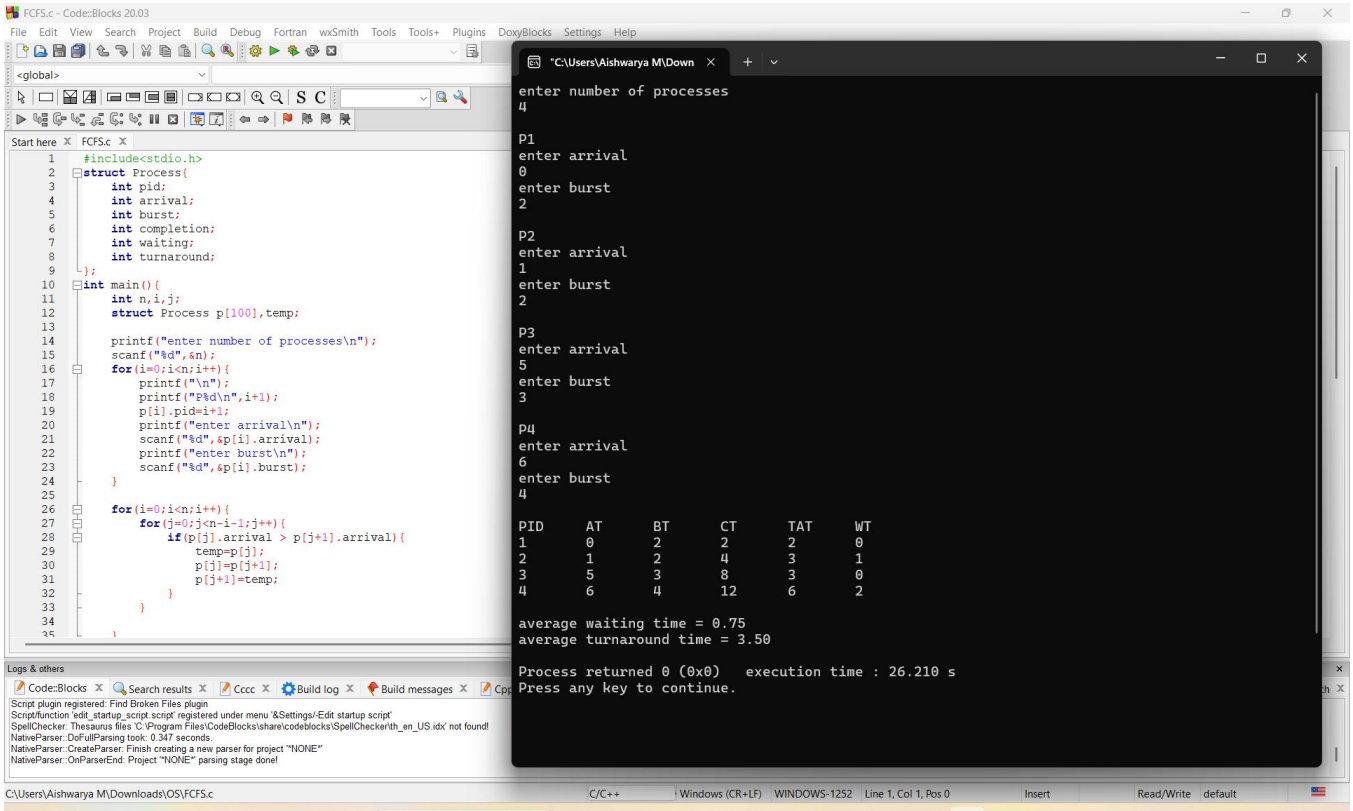
Code: =>FCFS:

```
#include<stdio.h>
struct Process{
    int pid;
    int arrival;
    int burst;
    int completion;
    int waiting;
    int turnaround;
};
int main(){
    int n,i,j;
    struct Process p[100],temp;

    printf("enter number of processes\n");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        printf("\n");
        printf("P%d\n",i+1);
        p[i].pid=i+1;
        printf("enter arrival\n");
        scanf("%d",&p[i].arrival);
        printf("enter burst\n");
        scanf("%d",&p[i].burst);
    }
```

```
for(i=0;i<n;i++){
    for(j=0;j<n-i-1;j++){
        if(p[j].arrival > p[j+1].arrival){
            temp=p[j];
            p[j]=p[j+1];
            p[j+1]=temp;
        }
    }

}
int current=0;
for(i=0;i<n;i++){
    if(current<p[i].arrival){
        current=p[i].arrival;
    }
    p[i].completion=current+p[i].burst;
    p[i].turnaround=p[i].completion-p[i].arrival;
    p[i].waiting=p[i].turnaround-p[i].burst;
    current =p[i].completion;
}

float totalWT = 0, totalTAT = 0;

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid,
            p[i].arrival,
            p[i].burst,
```

```
            p[i].completion,
            p[i].turnaround,
            p[i].waiting);

        totalWT += p[i].waiting;
        totalTAT += p[i].turnaround;
    }

    printf("\naverage waiting time = %.2f", totalWT / n);
    printf("\naverage turnaround time = %.2f\n", totalTAT / n);

    return 0;

}
```

Result:

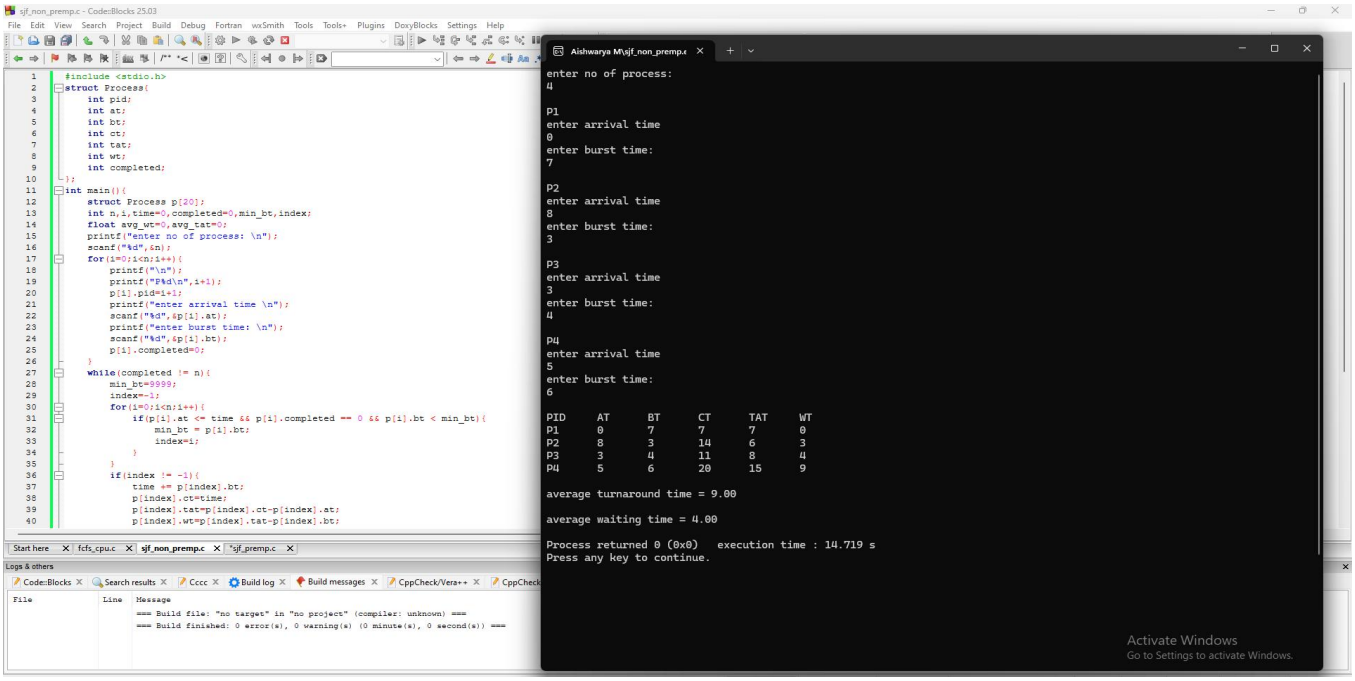


Code: => SJF(Non-preemptive):

```
#include <stdio.h>
struct Process{
    int pid;
    int at;
    int bt;
    int ct;
    int tat;
    int wt;
    int completed;
};

int main(){
    struct Process p[20];
    int n,i,time=0,completed=0,min_bt,index;
    float avg_wt=0,avg_tat=0;
    printf("enter no of process: \n");
    scanf("%d",&n);

    for(i=0;i<n;i++){
        printf("\n");
        printf("P%d\n",i+1);
        p[i].pid=i+1;
        printf("enter arrival time \n");
        scanf("%d",&p[i].at);
        printf("enter burst time: \n");
        scanf("%d",&p[i].bt);
        p[i].completed=0;
    }

    while(completed != n){
        min_bt=9999;
```

```
        index=-1;
        for(i=0;i<n;i++){
            if(p[i].at <= time && p[i].completed == 0 && p[i].bt < min_bt){
                min_bt = p[i].bt;
                index=i;
            }
        }

        if(index != -1){
            time += p[index].bt;
            p[index].ct=time;
            p[index].tat=p[index].ct-p[index].at;
            p[index].wt=p[index].tat-p[index].bt;

            avg_wt += p[index].wt;
            avg_tat += p[index].tat;

            p[index].completed =1;
            completed++;
        }else{
            time++;
        }
    }

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

    for(i=0;i<n;i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",p[i].pid,p[i].at,p[i].bt,p[i].ct,p[i].tat,p[i].wt);

    }
    printf("\naverage turnaround time = %.2f\n",avg_tat/n);
    printf("\naverage waiting time = %.2f\n",avg_wt/n);
    return 0;
}
```

Result:



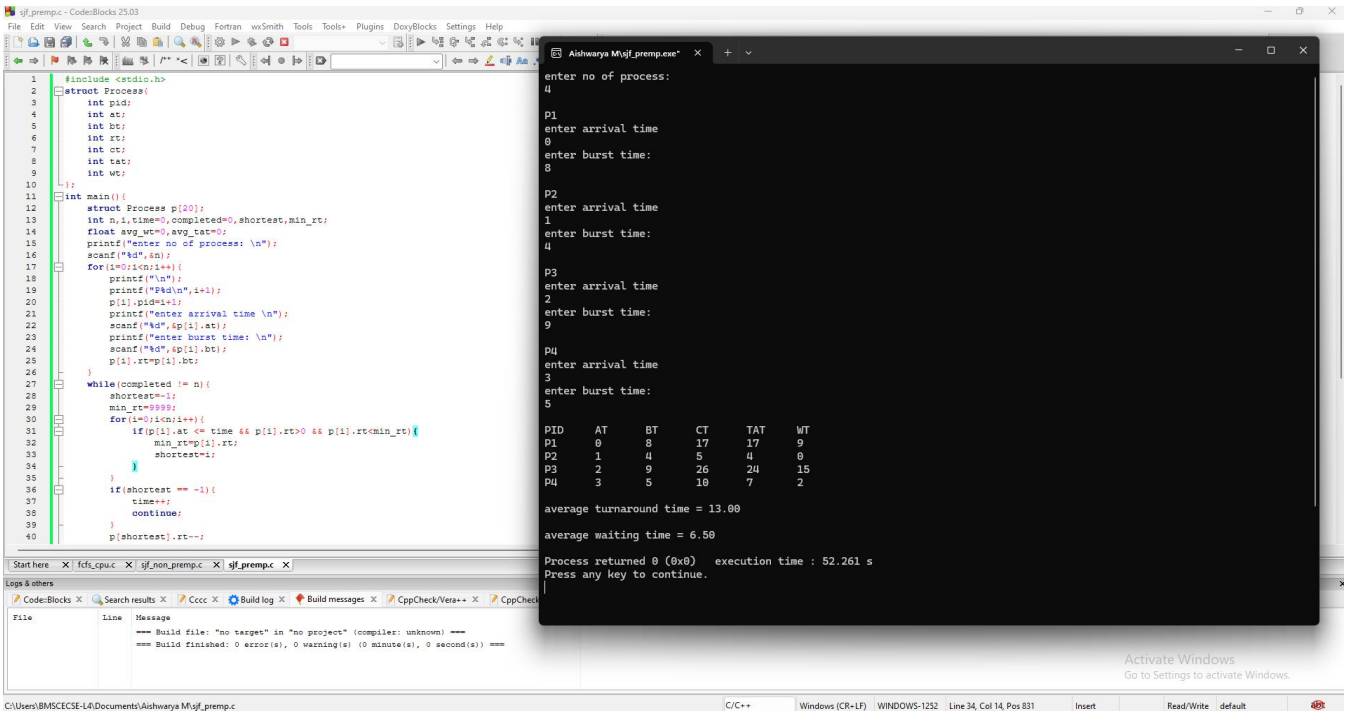
Code: =>SJF(preemptive):

```
#include <stdio.h>
struct Process{
    int pid;
    int at;
    int bt;
    int rt;
    int ct;
    int tat;
    int wt;
};
int main(){
    struct Process p[20];
    int n,i,time=0,completed=0,shortest,min_rt;
```

```
float avg_wt=0,avg_tat=0;
printf("enter no of process: \n");
scanf("%d",&n);
for(i=0;i<n;i++){
    printf("\n");
    printf("P%d\n",i+1);
    p[i].pid=i+1;
    printf("enter arrival time \n");
    scanf("%d",&p[i].at);
    printf("enter burst time: \n");
    scanf("%d",&p[i].bt);
    p[i].rt=p[i].bt;
}
while(completed != n){
    shortest=-1;
    min_rt=9999;
    for(i=0;i<n;i++){
        if(p[i].at <= time && p[i].rt>0 && p[i].rt<min_rt){
            min_rt=p[i].rt;
            shortest=i;
        }
    }
    if(shortest == -1){
        time++;
        continue;
    }
    p[shortest].rt--;
    time++;
    if(p[shortest].rt == 0){
        completed++;
```

```
            p[shortest].ct=time;
            p[shortest].tat=p[shortest].ct-p[shortest].at;
            p[shortest].wt=p[shortest].tat-p[shortest].bt;
            avg_tat+=p[shortest].tat;
            avg_wt += p[shortest].wt;
        }
    }
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",p[i].pid,p[i].at,p[i].bt,p[i].ct,p[i].tat,p[i].wt);

    }
    printf("\naverage turnaround time = %.2f\n",avg_tat/n);
    printf("\naverage waiting time = %.2f\n",avg_wt/n);
    return 0;
}
```

Result:



Program 1b)

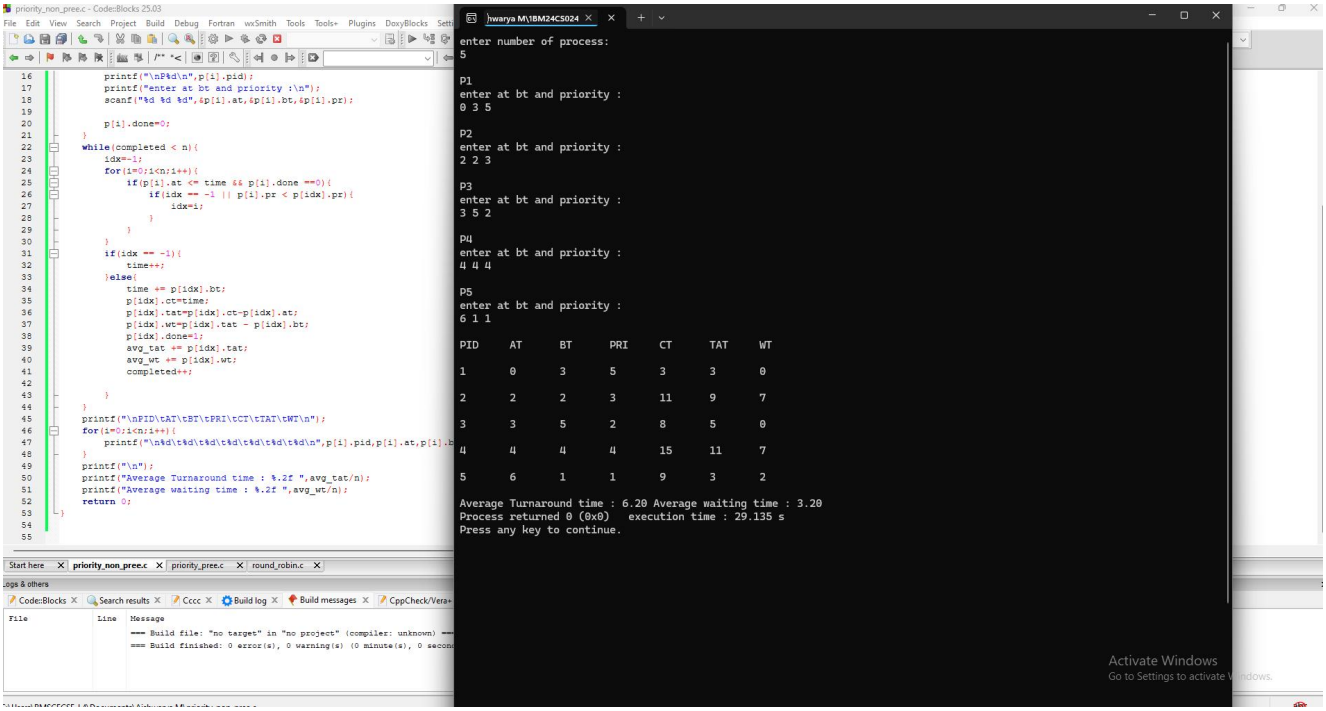
Question: Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.
→ Priority (pre-emptive & Non-pre-emptive)
→Round Robin (Experiment with different quantum sizes for RR algorithm)

Code : → Priority (Non-pre-emptive)

```
#include<stdio.h>
struct Process{
    int pid;
    int at,bt,pr;
    int ct,tat,wt;
    int done;
};
int main(){
    int n,i,time=0,completed=0,idx;
    float avg_tat=0,avg_wt=0;
    struct Process p[20];
    printf("enter number of process: \n");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        p[i].pid=i+1;
        printf("\nP%d\n",p[i].pid);
        printf("enter at bt and priority :\n");
        scanf("%d %d %d",&p[i].at,&p[i].bt,&p[i].pr);

        p[i].done=0;
    }
    while(completed < n){
        idx=-1;
        for(i=0;i<n;i++){
            if(p[i].at <= time && p[i].done ==0){
                if(idx == -1 || p[i].pr < p[idx].pr){
                    idx=i;
```

```
            }
        }
    }
    if(idx == -1){
        time++;
    }else{
        time += p[idx].bt;
        p[idx].ct=time;
        p[idx].tat=p[idx].ct-p[idx].at;
        p[idx].wt=p[idx].tat - p[idx].bt;
        p[idx].done=1;
        avg_tat += p[idx].tat;
        avg_wt += p[idx].wt;
        completed++;

    }
  }
  printf("\nPID\tAT\tBT\tPRI\tCT\tTAT\tWT\n");
  for(i=0;i<n;i++){
    printf("\n%d\t%d\t%d\t%d\t%d\t%d\t%d\n",p[i].pid,p[i].at,p[i].bt,p[i].pr,p[i].ct,p[i].tat,p[i].wt);
  }
  printf("\n");
  printf("Average Turnaround time : %.2f ",avg_tat/n);
  printf("Average waiting time : %.2f ",avg_wt/n);
  return 0;
}
```

Result:



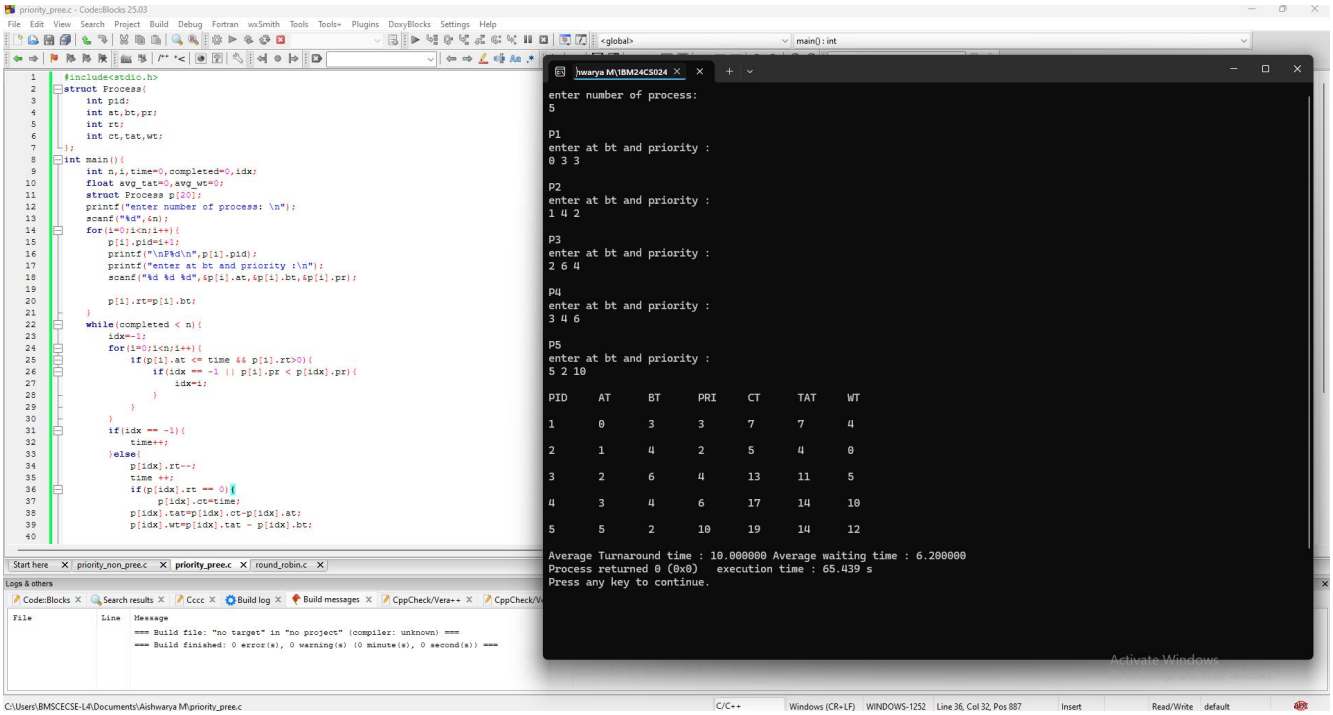
Code : → Priority (pre-emptive)

```
#include<stdio.h>
struct Process{
    int pid;
    int at,bt,pr;
    int rt;
    int ct,tat,wt;
};
int main(){
    int n,i,time=0,completed=0,idx;
    float avg_tat=0,avg_wt=0;
    struct Process p[20];
    printf("enter number of process: \n");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        p[i].pid=i+1;
        printf("\nP%d\n",p[i].pid);
        printf("enter at bt and priority :\n");
```

```
    scanf("%d %d %d",&p[i].at,&p[i].bt,&p[i].pr);

    p[i].rt=p[i].bt;
}
while(completed < n){
    idx=-1;
    for(i=0;i<n;i++){
        if(p[i].at <= time && p[i].rt>0){
            if(idx == -1 || p[i].pr < p[idx].pr){
                idx=i;
            }
        }
    }
    if(idx == -1){
        time++;
    }else{
        p[idx].rt--;
        time ++;
        if(p[idx].rt == 0){
             p[idx].ct=time;
        p[idx].tat=p[idx].ct-p[idx].at;
        p[idx].wt=p[idx].tat - p[idx].bt;

        avg_tat += p[idx].tat;
        avg_wt += p[idx].wt;
        completed++;
        }
    }
}
printf("\nPID\tAT\tBT\tPRI\tCT\tTAT\tWT\n");
for(i=0;i<n;i++){
    printf("\n%d\t%d\t%d\t%d\t%d\t%d\t%d\n",p[i].pid,p[i].at,p[i].bt,p[i].pr,p[i].ct,p[i].tat,p[i].wt);
}
printf("\n");
```

```
    printf("Average Turnaround time : %f ",avg_tat/n);
    printf("Average waiting time : %f ",avg_wt/n);

    return 0;
}
```

Result:



Code : → Round Robin

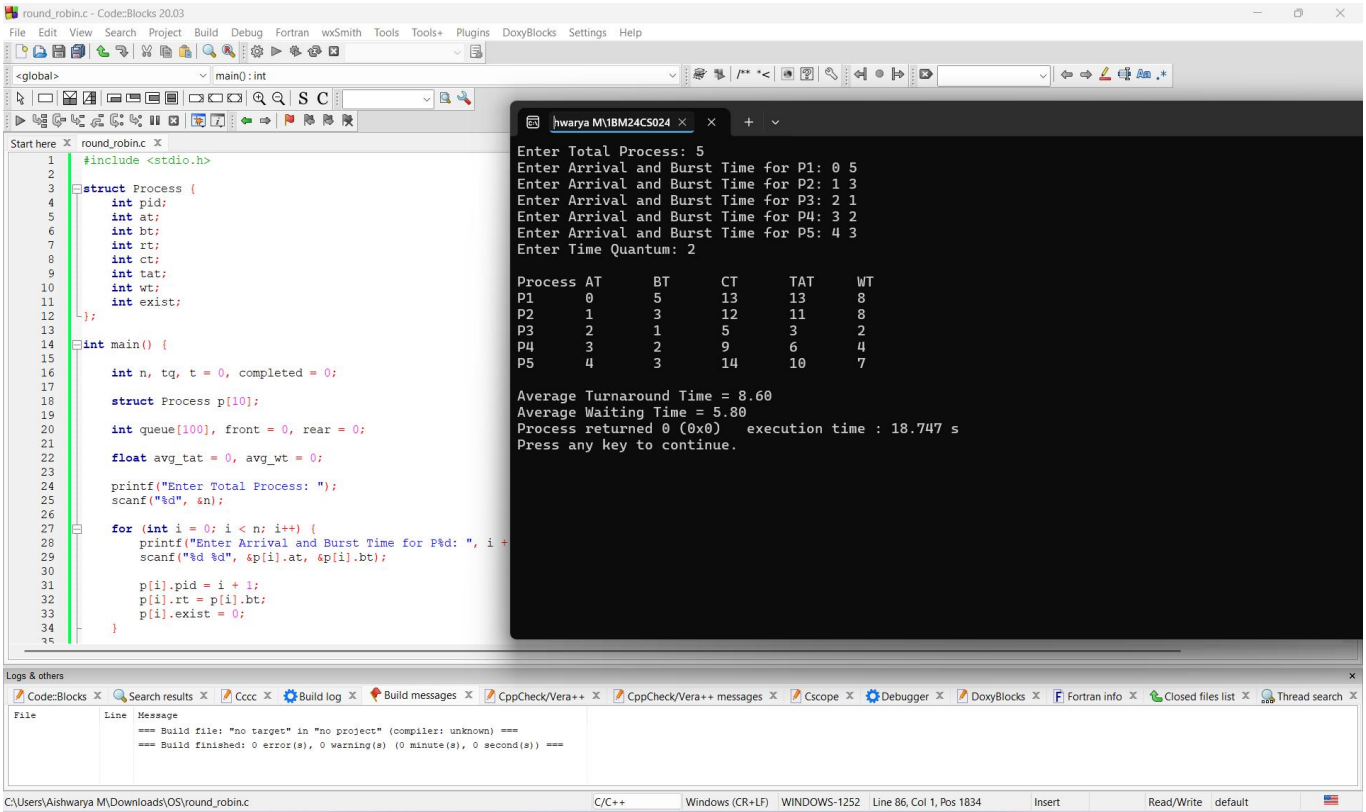
```
#include <stdio.h>

struct Process {
    int pid;
    int at;
    int bt;
    int rt;
    int ct;
    int tat;
    int wt;
```

```
    int exist;
};
int main() {
    int n, tq, t = 0, completed = 0;
    struct Process p[10];
    int queue[100], front = 0, rear = 0;
    float avg_tat = 0, avg_wt = 0;

    printf("Enter Total Process: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter Arrival and Burst Time for P%d: ", i + 1);
        scanf("%d %d", &p[i].at, &p[i].bt);
        p[i].pid = i + 1;
        p[i].rt = p[i].bt;
        p[i].exist = 0;
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    for (int i = 0; i < n; i++) {
        if (p[i].at <= t) {
            queue[rear++] = i;
            p[i].exist = 1;
        }
    }

    while (front < rear) {

        int i = queue[front++];

        if (p[i].rt > tq) {
```

```
            t += tq;
            p[i].rt -= tq;
        }
        else {
            t += p[i].rt;
            p[i].rt = 0;
            completed++;

            p[i].ct = t;
            p[i].tat = p[i].ct - p[i].at;
            p[i].wt = p[i].tat - p[i].bt;
        }

        for (int j = 0; j < n; j++) {
            if (p[j].at <= t && p[j].exist == 0) {
                queue[rear++] = j;
                p[j].exist = 1;
            }
        }
        if (p[i].rt > 0) {
            queue[rear++] = i;
        }
        if (front == rear && completed < n) {
            for (int j = 0; j < n; j++) {
                if (p[j].exist == 0) {
                    t = p[j].at;
                    queue[rear++] = j;
                    p[j].exist = 1;
                    break;
                }
            }
        }
    }
```

```
    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].pid, p[i].at, p[i].bt,
               p[i].ct, p[i].tat, p[i].wt);

        avg_tat += p[i].tat;
        avg_wt += p[i].wt;
    }
    avg_tat /= n;
    avg_wt /= n;
    printf("\nAverage Turnaround Time = %.2f", avg_tat);
    printf("\nAverage Waiting Time = %.2f", avg_wt);

    return 0;
}
```

Result:



Program 2)

Question: Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code:

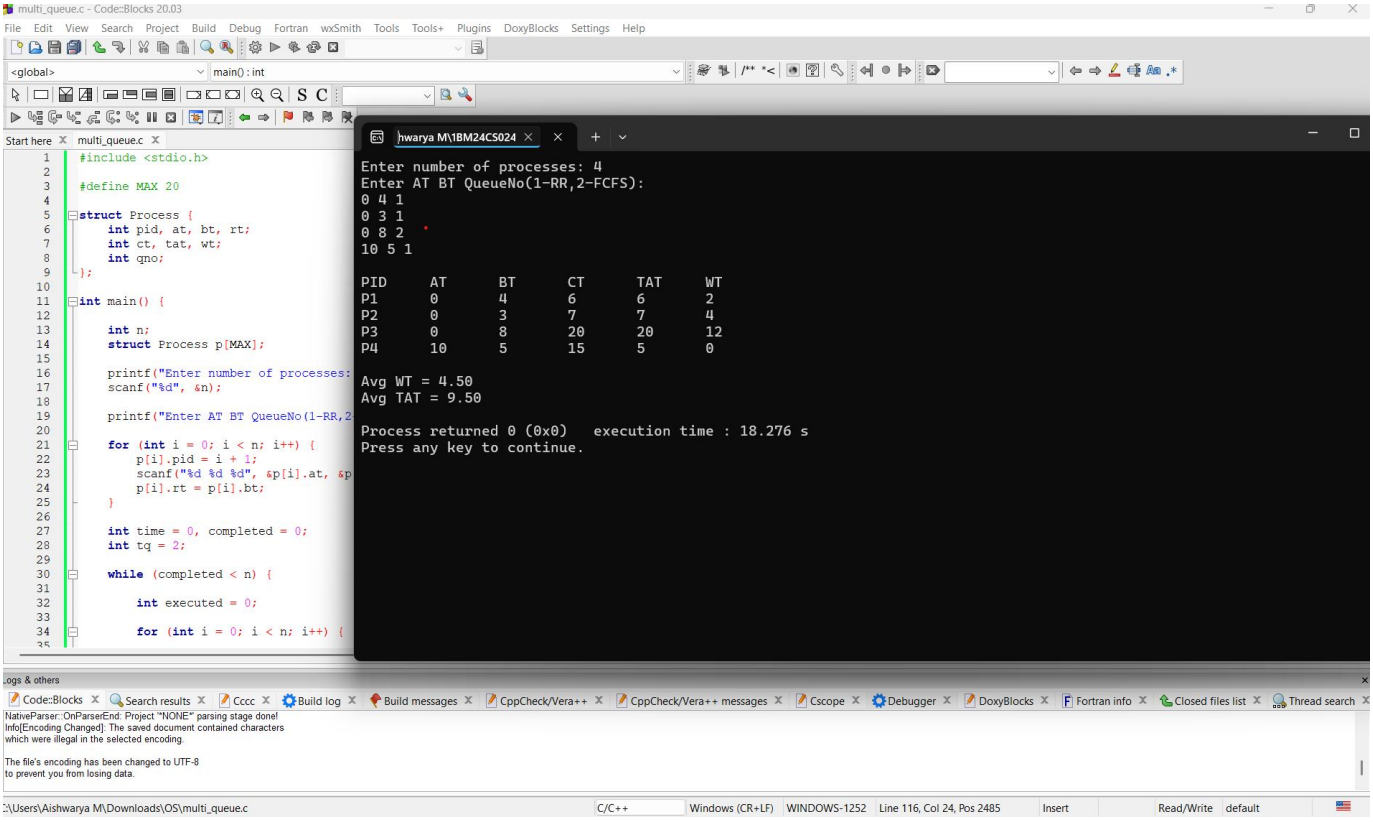
```
#include <stdio.h>
#define MAX 20
struct Process {
    int pid, at, bt, rt;
    int ct, tat, wt;
    int qno;
};
int main() {
    int n;
    struct Process p[MAX];
    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter AT BT QueueNo(1-RR,2-FCFS):\n");
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].qno);
        p[i].rt = p[i].bt;
    }
    int time = 0, completed = 0;
    int tq = 2;

    while (completed < n) {
        int executed = 0;
        for (int i = 0; i < n; i++) {

            if (p[i].qno == 1 &&
```

```
        p[i].rt > 0 &&
        p[i].at <= time) {

        executed = 1;
        int run = (p[i].rt > tq) ? tq : p[i].rt;

        time += run;
        p[i].rt -= run;

        if (p[i].rt == 0) {
            p[i].ct = time;
            completed++;
        }
    }
}

if (executed) continue;
int idx = -1;
int earliest = 99999;

for (int i = 0; i < n; i++) {
    if (p[i].qno == 2 &&
        p[i].rt > 0 &&
        p[i].at <= time) {

        if (p[i].at < earliest) {
            earliest = p[i].at;
            idx = i;
        }
    }
}
if (idx != -1) {

    executed = 1;
```

```
        while (p[idx].rt > 0) {
            int q1_ready = 0;

            for (int j = 0; j < n; j++) {
                if (p[j].qno == 1 &&
                    p[j].rt > 0 &&
                    p[j].at <= time) {
                    q1_ready = 1;
                    break;
                }
            }

            if (q1_ready)
                break;
            time++;
            p[idx].rt--;
        }

        if (p[idx].rt == 0) {
            p[idx].ct = time;
            completed++;
        }
    }
    if (!executed)
        time++;
}

float avg_wt = 0, avg_tat = 0;

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for (int i = 0; i < n; i++) {
    p[i].tat = p[i].ct - p[i].at;
```

```
        p[i].wt = p[i].tat - p[i].bt;
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].pid,
               p[i].at,
               p[i].bt,
               p[i].ct,
               p[i].tat,
               p[i].wt);
        avg_wt += p[i].wt;
        avg_tat += p[i].tat;
    }
    printf("\nAvg WT = %.2f", avg_wt / n);
    printf("\nAvg TAT = %.2f\n", avg_tat / n);
    return 0;
}
```

Result:



Program 3)

Question: Write a C program to simulate Real-Time CPU Scheduling algorithms:

a. Rate- Monotonic
b. Earliest-deadline First
c. Proportional scheduling

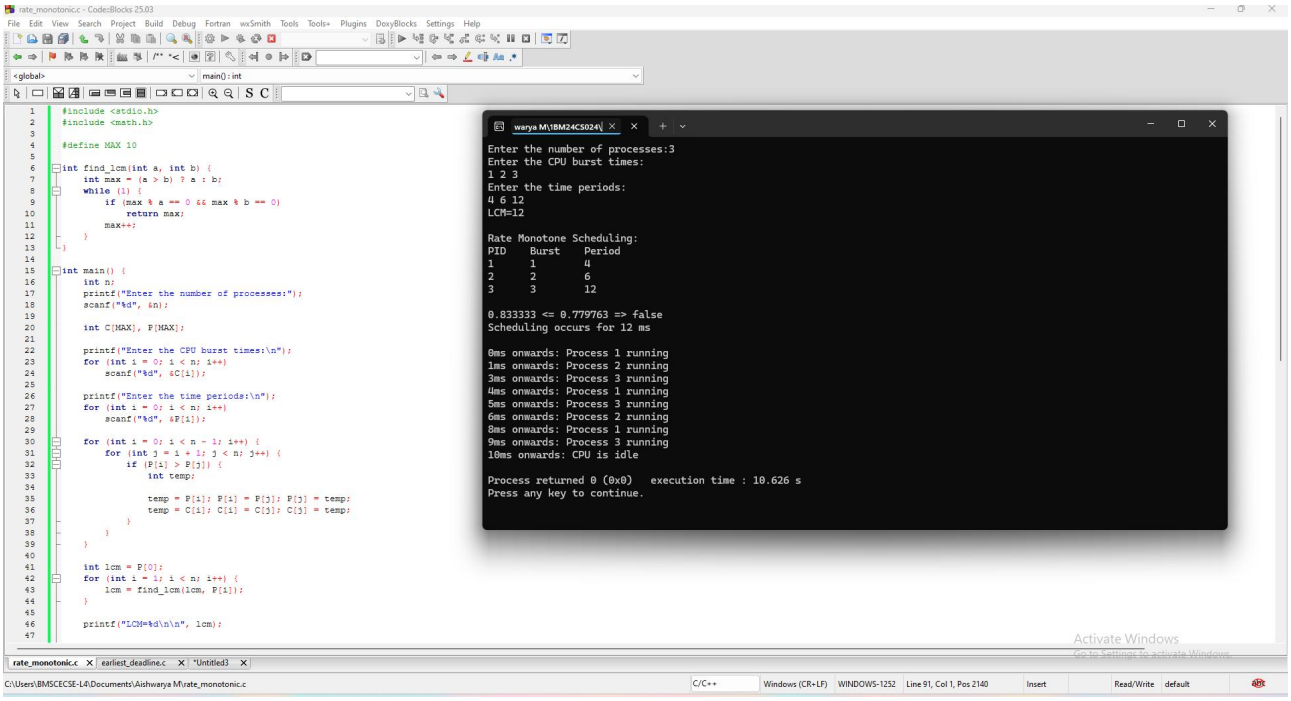
Code: a. Rate- Monotonic

```
#include <stdio.h>
#include <math.h>

#define MAX 10

int find_lcm(int a, int b) {
   int max = (a > b) ? a : b;
   while (1) {
      if (max % a == 0 && max % b == 0)
         return max;
      max++;
   }
}

int main() {
   int n;
   printf("Enter the number of processes:");
   scanf("%d", &n);

   int C[MAX], P[MAX];

   printf("Enter the CPU burst times:\n");
   for (int i = 0; i < n; i++)
      scanf("%d", &C[i]);

   printf("Enter the time periods:\n");
   for (int i = 0; i < n; i++)
      scanf("%d", &P[i]);

   for (int i = 0; i < n - 1; i++) {
      for (int j = i + 1; j < n; j++) {
         if (P[i] > P[j]) {
            int temp;

            temp = P[i]; P[i] = P[j]; P[j] = temp;
            temp = C[i]; C[i] = C[j]; C[j] = temp;
         }
      }
```

```
}

int lcm = P[0];
for (int i = 1; i < n; i++) {
   lcm = find_lcm(lcm, P[i]);
}

printf("LCM=%d\n\n", lcm);

printf("Rate Monotone Scheduling:\n");
printf("PID    Burst    Period\n");
for (int i = 0; i < n; i++) {
   printf("%d      %d        %d\n", i + 1, C[i], P[i]);
}

float U = 0;
for (int i = 0; i < n; i++) {
   U += (float)C[i] / P[i];
}

float bound = n * (pow(2, (float)1 / n) - 1);

printf("\n%f <= %f => %s\n", U, bound, (U <= bound) ? "true" : "false");

printf("Scheduling occurs for %d ms\n\n", lcm);

int remaining[MAX] = {0};
int current = -1;

for (int t = 0; t < lcm; t++) {

   for (int i = 0; i < n; i++) {
      if (t % P[i] == 0) {
         remaining[i] = C[i];
      }
   }

   int next = -1;
   for (int i = 0; i < n; i++) {
      if (remaining[i] > 0) {
         next = i;
         break;
      }
   }

   if (next != current) {
      if (next == -1)
```

```
            printf("%dms onwards: CPU is idle\n", t);
        else
            printf("%dms onwards: Process %d running\n", t, next + 1);
        current = next;
    }

    if (next != -1) {
        remaining[next]--;
    }
  }

  return 0;
}
```

Result:



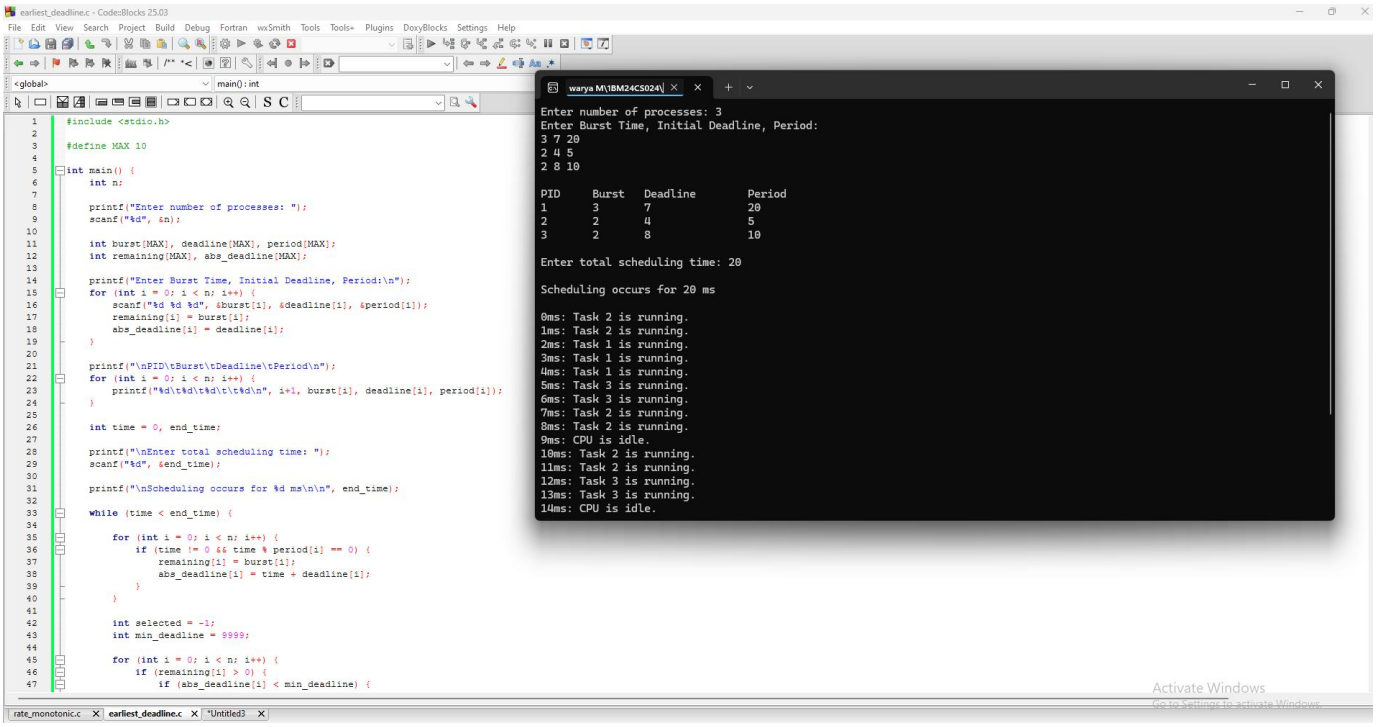
Code: b. Earliest-deadline First

```
#include <stdio.h>

#define MAX 10

int main() {
    int n;

    printf("Enter number of processes: ");
```

```
scanf("%d", &n);

int burst[MAX], deadline[MAX], period[MAX];
int remaining[MAX], abs_deadline[MAX];

printf("Enter Burst Time, Initial Deadline, Period:\n");
for (int i = 0; i < n; i++) {
   scanf("%d %d %d", &burst[i], &deadline[i], &period[i]);
   remaining[i] = burst[i];
   abs_deadline[i] = deadline[i];
}

printf("\nPID\tBurst\tDeadline\tPeriod\n");
for (int i = 0; i < n; i++) {
   printf("%d\t%d\t%d\t\t%d\n", i+1, burst[i], deadline[i], period[i]);
}

int time = 0, end_time;

printf("\nEnter total scheduling time: ");
scanf("%d", &end_time);

printf("\nScheduling occurs for %d ms\n\n", end_time);

while (time < end_time) {

   for (int i = 0; i < n; i++) {
      if (time != 0 && time % period[i] == 0) {
         remaining[i] = burst[i];
         abs_deadline[i] = time + deadline[i];
      }
   }

   int selected = -1;
   int min_deadline = 9999;

   for (int i = 0; i < n; i++) {
      if (remaining[i] > 0) {
         if (abs_deadline[i] < min_deadline) {
            min_deadline = abs_deadline[i];
            selected = i;
         }
      }
   }

   if (selected == -1) {
      printf("%dms: CPU is idle.\n", time);
```

```
        } else {
            printf("%dms: Task %d is running.\n", time, selected + 1);
            remaining[selected]--; // execute
        }

        time++;
    }

    return 0;
}
```

Result:



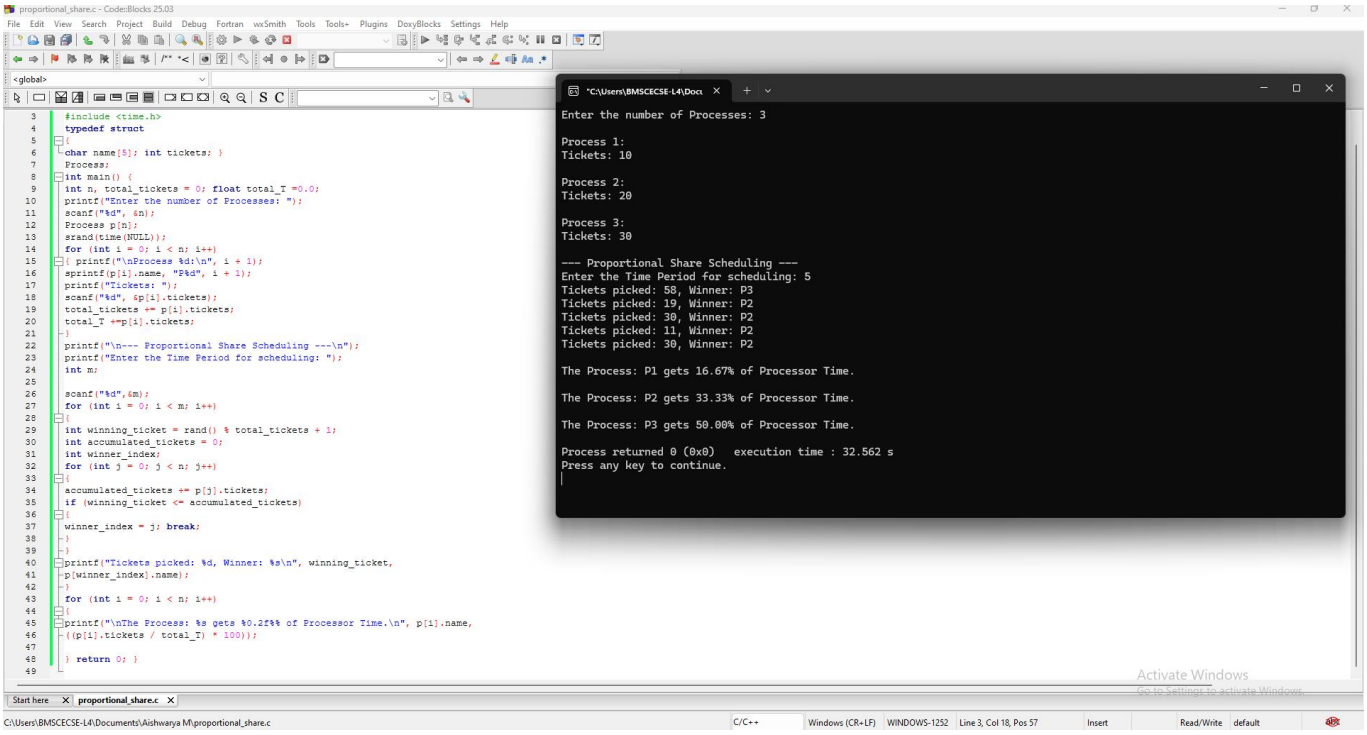
Code: c. Proportional scheduling

```
#include<stdio.h>
#include <stdlib.h>
#include <time.h>
typedef struct
{
char name[5]; int tickets; }
Process;
```

```
int main() {
int n, total_tickets = 0; float total_T =0.0;
printf("Enter the number of Processes: ");
scanf("%d", &n);
Process p[n];
srand(time(NULL));
for (int i = 0; i < n; i++)
{ printf("\nProcess %d:\n", i + 1);
sprintf(p[i].name, "P%d", i + 1);
printf("Tickets: ");
scanf("%d", &p[i].tickets);
total_tickets += p[i].tickets;
total_T +=p[i].tickets;
}
printf("\n--- Proportional Share Scheduling ---\n");
printf("Enter the Time Period for scheduling: ");
int m;

scanf("%d",&m);
for (int i = 0; i < m; i++)
{
int winning_ticket = rand() % total_tickets + 1;
int accumulated_tickets = 0;
int winner_index;
for (int j = 0; j < n; j++)
{
accumulated_tickets += p[j].tickets;
if (winning_ticket <= accumulated_tickets)
{
winner_index = j; break;
}
}
printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
p[winner_index].name);
```

```
}
for (int i = 0; i < n; i++)
{
printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,
((p[i].tickets / total_T) * 100));

}
return 0;
}
```

Result:



Program 4)

Question: Write a C program to simulate:
a) Producer-consumer problem using semaphores
b) Dining-Philosopher's problem

Code :a. Producer-consumer problem using semaphores

```
#include <stdio.h>
#include <stdlib.h>

int *buffer;
int SIZE;
int in = 0, out = 0;
int mutex = 1;
int full = 0;
int empty;

void wait(int *s)
{
   (*s)--;
}

void signal(int *s)
{
   (*s)++;
}

void display()
{
   printf("\nBuffer: [ ");
   int i = out;
   for (int c = 0; c < full; c++)
   {
      printf("%d ", buffer[i]);
      i = (i + 1) % SIZE;
   }
   printf("]\n");
}

void producer()
{
   int item;

   printf("\n\nProducer tries to enter\n");
```

```
    wait(&mutex);
    printf("wait(mutex) -> mutex = %d\n", mutex);

    if (mutex < 0)
    {
        printf("Producer BLOCKED\n");
        signal(&mutex);
        return;
    }

    if (empty == 0)
    {
        printf("Buffer FULL, cannot produce\n");
        signal(&mutex);
        return;
    }

    wait(&empty);

    printf("\nProducer ENTERED critical section\n");

    printf("\nEnter item: ");
    scanf("%d", &item);

    buffer[in] = item;
    printf("\nProduced %d at position %d\n", item, in);

    in = (in + 1) % SIZE;

    printf("\nProducer leaving critical section\n");

    signal(&mutex);
    signal(&full);

    printf("signal(mutex) -> mutex = %d\n", mutex);
}

void consumer()
{
    int item;

    printf("\n\nConsumer tries to enter\n");

    wait(&mutex);
    printf("wait(mutex) -> mutex = %d\n", mutex);

    if (mutex < 0)
```

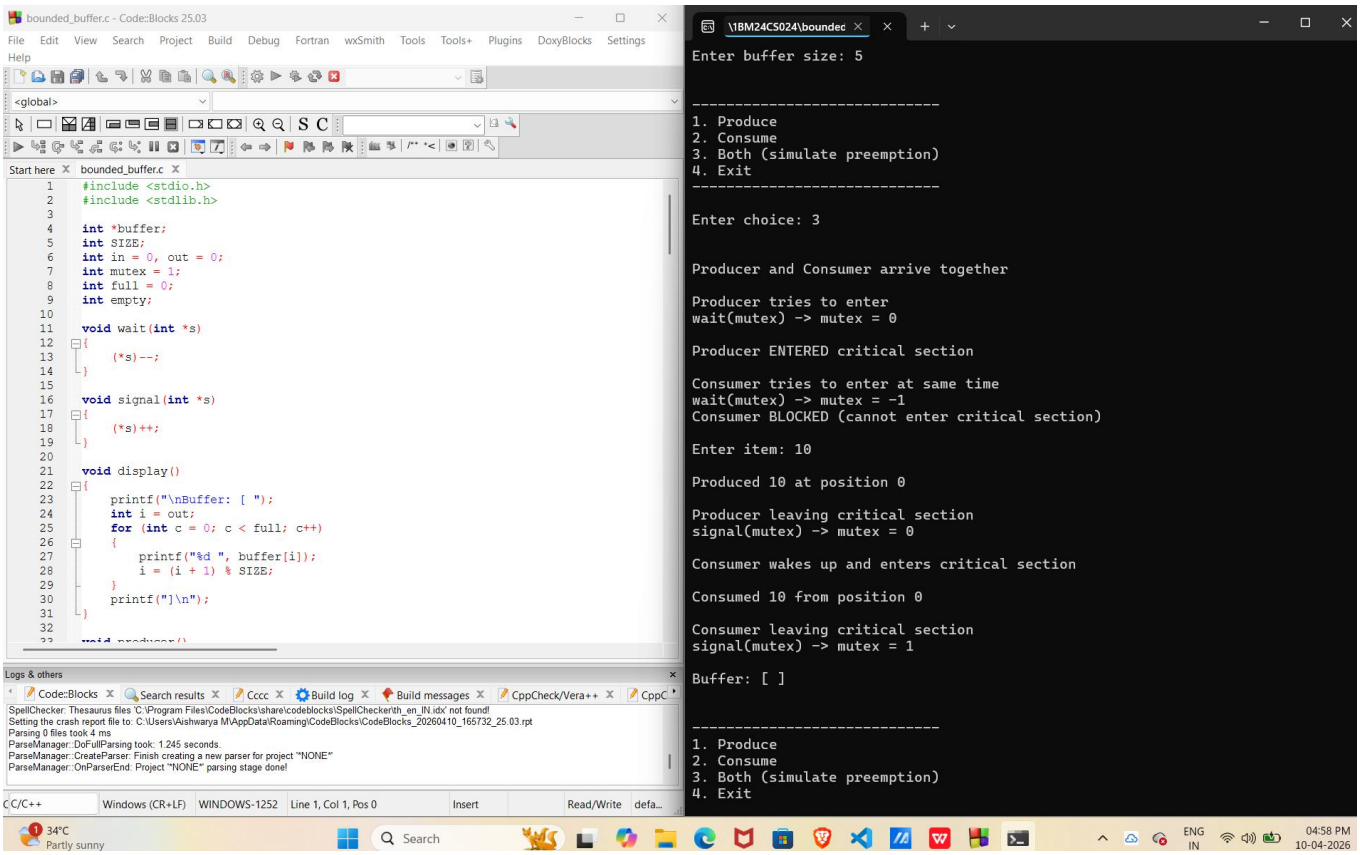
```
    {
        printf("Consumer BLOCKED\n");
        signal(&mutex);
        return;
    }

    if (full == 0)
    {
        printf("Buffer EMPTY, cannot consume\n");
        signal(&mutex);
        return;
    }

    wait(&full);

    printf("\nConsumer ENTERED critical section\n");

    item = buffer[out];
    printf("\nConsumed %d from position %d\n", item, out);

    out = (out + 1) % SIZE;

    printf("\nConsumer leaving critical section\n");

    signal(&mutex);
    signal(&empty);

    printf("signal(mutex) -> mutex = %d\n", mutex);
}

void both()
{
    int item;

    printf("\n\nProducer and Consumer arrive together\n");

    printf("\nProducer tries to enter\n");
    wait(&mutex);
    printf("wait(mutex) -> mutex = %d\n", mutex);

    printf("\nProducer ENTERED critical section\n");

    printf("\nConsumer tries to enter at same time\n");
    wait(&mutex);
    printf("wait(mutex) -> mutex = %d\n", mutex);

    if (mutex < 0)
```

```
{
   printf("Consumer BLOCKED (cannot enter critical section)\n");
}

if (empty == 0)
{
   printf("\nBuffer FULL, Producer cannot proceed\n");
}
else
{
   wait(&empty);

   printf("\nEnter item: ");
   scanf("%d", &item);

   buffer[in] = item;
   printf("\nProduced %d at position %d\n", item, in);

   in = (in + 1) % SIZE;
}

printf("\nProducer leaving critical section\n");

signal(&mutex);
printf("signal(mutex) -> mutex = %d\n", mutex);

signal(&full);

printf("\nConsumer wakes up and enters critical section\n");

if (full == 0)
{
   printf("Buffer EMPTY, cannot consume\n");
   signal(&mutex);
   return;
}

wait(&full);

item = buffer[out];
printf("\nConsumed %d from position %d\n", item, out);

out = (out + 1) % SIZE;

printf("\nConsumer leaving critical section\n");

signal(&mutex);
```

```
    signal(&empty);

    printf("signal(mutex) -> mutex = %d\n", mutex);
}

int main()
{
    int choice;

    printf("Enter buffer size: ");
    scanf("%d", &SIZE);

    buffer = (int *)malloc(SIZE * sizeof(int));
    empty = SIZE;

    while (1)
    {
        printf("\n\n----------------------------\n");
        printf("1. Produce\n2. Consume\n3. Both (simulate preemption)\n4. Exit\n");
        printf("----------------------------\n");

        printf("\nEnter choice: ");
        scanf("%d", &choice);

        if (choice == 1)
        {
            producer();
            display();
        }
        else if (choice == 2)
        {
            consumer();
            display();
        }
        else if (choice == 3)
        {
            both();
            display();
        }
        else if (choice == 4)
        {
            free(buffer);
            return 0;
        }
        else
        {
            printf("\nInvalid choice\n");
```

```
    }
  }
}
```

Result:



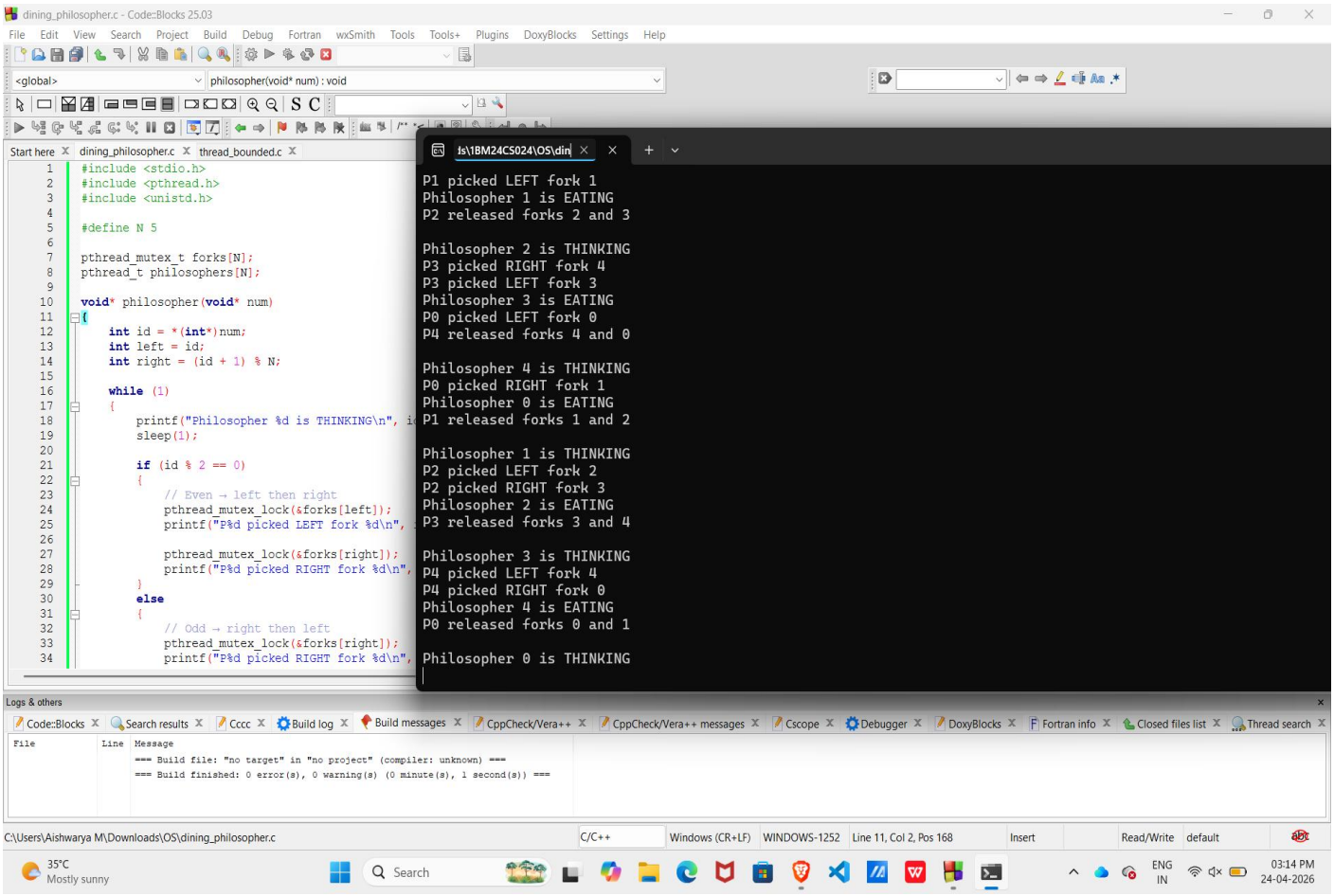
Code: b. Dining-Philosopher's problem

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5

pthread_mutex_t forks[N];
pthread_t philosophers[N];

void* philosopher(void* num)
{
   int id = *(int*)num;
   int left = id;
   int right = (id + 1) % N;
```

```
    while (1)
    {
        printf("Philosopher %d is THINKING\n", id);
        sleep(1);

        if (id % 2 == 0)
        {
            // Even → left then right
            pthread_mutex_lock(&forks[left]);
            printf("P%d picked LEFT fork %d\n", id, left);

            pthread_mutex_lock(&forks[right]);
            printf("P%d picked RIGHT fork %d\n", id, right);
        }
        else
        {
            // Odd → right then left
            pthread_mutex_lock(&forks[right]);
            printf("P%d picked RIGHT fork %d\n", id, right);

            pthread_mutex_lock(&forks[left]);
            printf("P%d picked LEFT fork %d\n", id, left);
        }

        printf("Philosopher %d is EATING\n", id);
        sleep(2);

        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);

        printf("P%d released forks %d and %d\n\n", id, left, right);
    }

    return NULL;
}

int main()
{
    int ids[N];

    for (int i = 0; i < N; i++)
    {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }
```

```
    for (int i = 0; i < N; i++)
    {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < N; i++)
    {
        pthread_join(philosophers[i], NULL);
    }

    for (int i = 0; i < N; i++)
    {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}
```

Result:



Program 5)

Question: Write a C program to simulate
a) Banker's algorithm for the purpose of deadlock avoidance.
b) Deadlock Detection

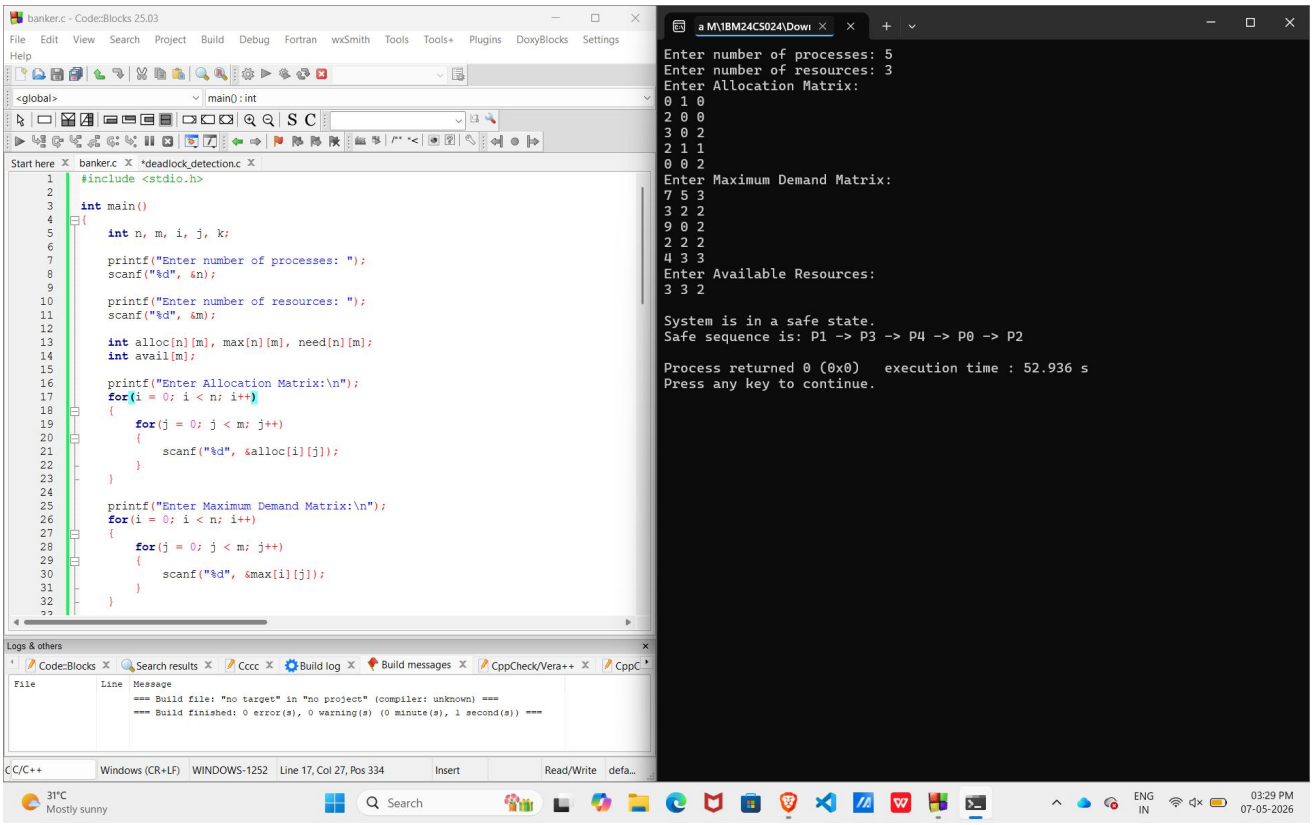
Code :a. Banker's algorithm for the purpose of deadlock avoidance.

```
#include <stdio.h>

int main()
{
   int n, m, i, j, k;

   printf("Enter number of processes: ");
   scanf("%d", &n);
   printf("Enter number of resources: ");
   scanf("%d", &m);

   int alloc[n][m], max[n][m], need[n][m];
   int avail[m];

   printf("Enter Allocation Matrix:\n");
   for(i = 0; i < n; i++)
   {
      for(j = 0; j < m; j++)
      {
         scanf("%d", &alloc[i][j]);
      }
   }

   printf("Enter Maximum Demand Matrix:\n");
   for(i = 0; i < n; i++)
   {
      for(j = 0; j < m; j++)
      {
         scanf("%d", &max[i][j]);
      }
   }

   printf("Enter Available Resources:\n");
   for(i = 0; i < m; i++)
   {
      scanf("%d", &avail[i]);
   }

   for(i = 0; i < n; i++)
   {
```

```
    for(j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

int finish[n], safeSeq[n];
for(i = 0; i < n; i++)
{
    finish[i] = 0;
}

int count = 0;

while(count < n)
{
    int found = 0;
    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int flag = 1;
            for(j = 0; j < m; j++)
            {
                if(need[i][j] > avail[j])
                {
                    flag = 0;
                    break;
                }
            }

            if(flag)
            {
                for(k = 0; k < m; k++)
                {
                    avail[k] += alloc[i][k];
                }

                safeSeq[count] = i;
                count++;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0)
```

```
        {
            printf("\nSystem is not in safe state.\n");
            return 0;
        }
    }

    printf("\nSystem is in a safe state.\n");
    printf("Safe sequence is: ");

    for(i = 0; i < n; i++)
    {
        printf("P%d", safeSeq[i]);

        if(i != n - 1)
            printf(" -> ");
    }

    printf("\n");

    return 0;
}
```

Result:



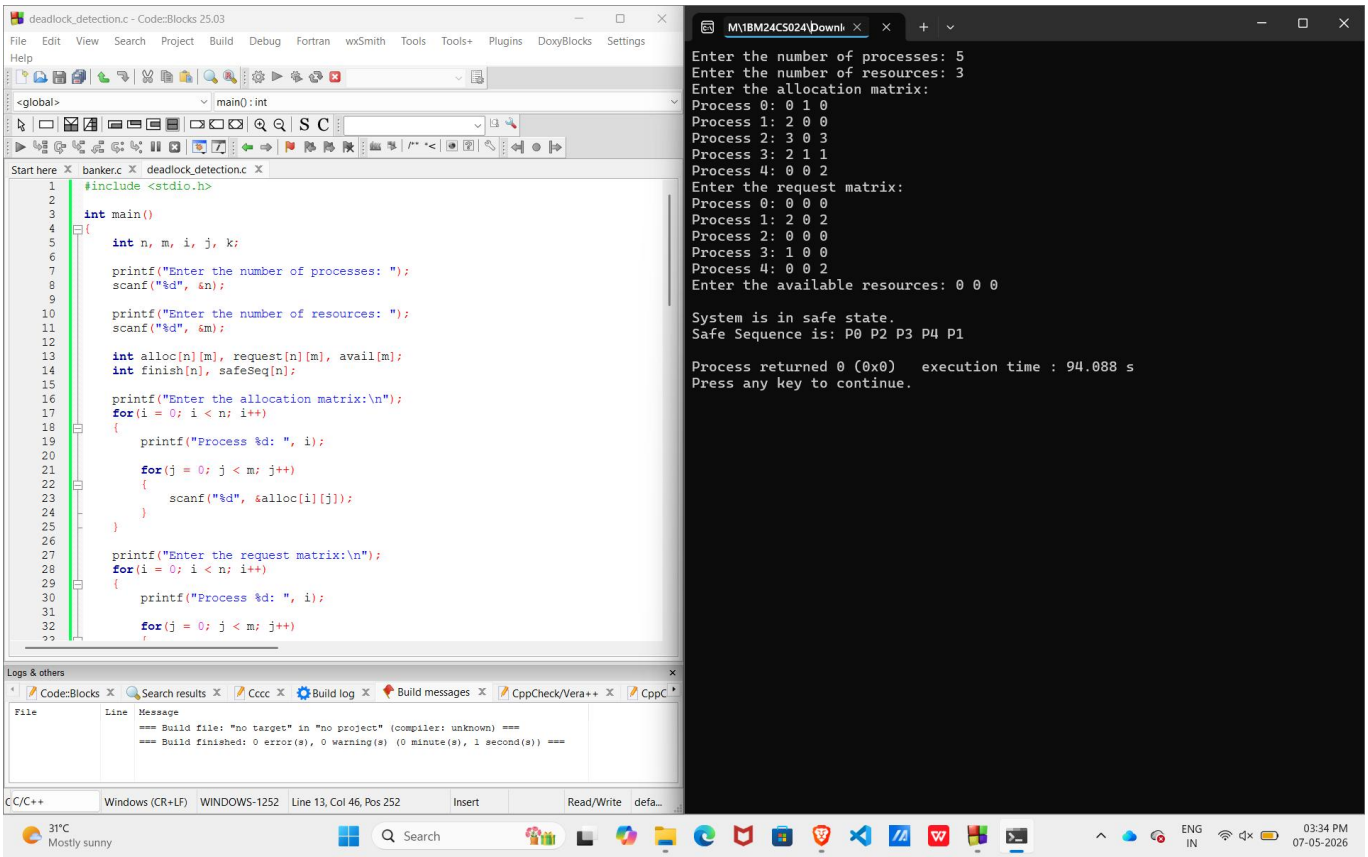
Code: b. Deadlock Detection

```
#include <stdio.h>

int main()
{
   int n, m, i, j, k;
   printf("Enter the number of processes: ");
   scanf("%d", &n);

   printf("Enter the number of resources: ");
   scanf("%d", &m);

   int alloc[n][m], request[n][m], avail[m];
   int finish[n], safeSeq[n];
   printf("Enter the allocation matrix:\n");
   for(i = 0; i < n; i++)
   {
      printf("Process %d: ", i);
      for(j = 0; j < m; j++)
      {
         scanf("%d", &alloc[i][j]);
      }
   }

   printf("Enter the request matrix:\n");
   for(i = 0; i < n; i++)
   {
      printf("Process %d: ", i);
      for(j = 0; j < m; j++)
      {
         scanf("%d", &request[i][j]);
      }
   }

   printf("Enter the available resources: ");
   for(i = 0; i < m; i++)
   {
      scanf("%d", &avail[i]);
   }
   for(i = 0; i < n; i++)
   {
      finish[i] = 0;
   }

   int count = 0;
```

```
while(count < n)
{
   int found = 0;
   for(i = 0; i < n; i++)
   {
      if(finish[i] == 0)
      {
         int flag = 1;

         for(j = 0; j < m; j++)
         {
            if(request[i][j] > avail[j])
            {
               flag = 0;
               break;
            }
         }

         if(flag)
         {
            for(k = 0; k < m; k++)
            {
               avail[k] += alloc[i][k];
            }
            safeSeq[count] = i;
            count++;
            finish[i] = 1;
            found = 1;
         }
      }
   }

   if(found == 0)
   {
      break;
   }
}

if(count == n)
{
   printf("\nSystem is in safe state.\n");
   printf("Safe Sequence is: ");

   for(i = 0; i < n; i++)
   {
      printf("P%d", safeSeq[i]);
```

```
            if(i != n - 1)
                printf(" ");
        }
        printf("\n");
    }
    else
    {
        printf("\nDeadlock detected.\nProcesses in deadlock are: ");

        for(i = 0; i < n; i++)
        {
            if(finish[i] == 0)
            {
                printf("P%d ", i);
            }
        }

        printf("\n");
    }
    return 0;
}
```

Result:



Program 6)

Question: Write a C program to simulate the following contiguous memory allocation techniques.
a) Worst-fit b) Best-fit c) First-fit

Code:

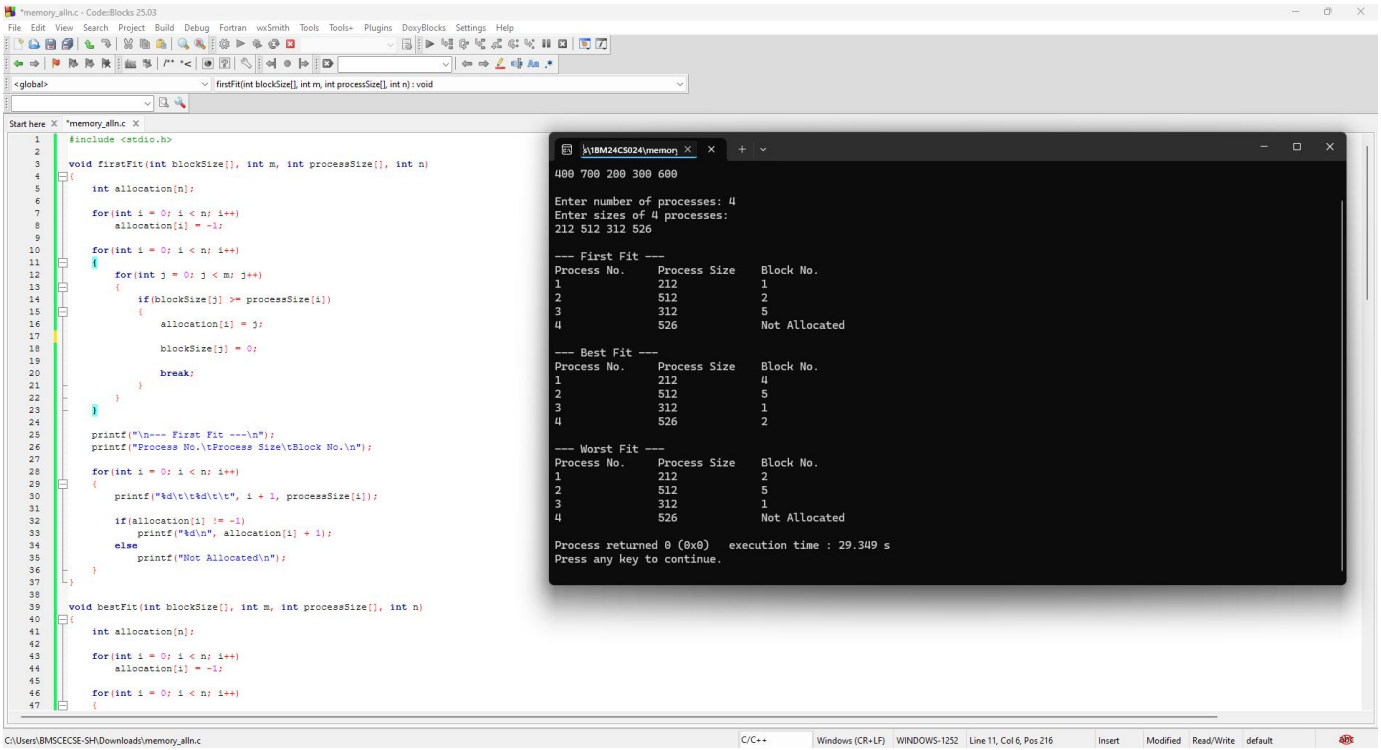
```
#include <stdio.h>

void firstFit(int blockSize[], int m, int processSize[], int n)
{
   int allocation[n];

   for(int i = 0; i < n; i++)
      allocation[i] = -1;

   for(int i = 0; i < n; i++)
   {
      for(int j = 0; j < m; j++)
      {
         if(blockSize[j] >= processSize[i])
         {
            allocation[i] = j;

            blockSize[j] = 0;

            break;
         }
      }
   }

   printf("\n--- First Fit ---\n");
   printf("Process No.\tProcess Size\tBlock No.\n");

   for(int i = 0; i < n; i++)
   {
      printf("%d\t\t%d\t\t", i + 1, processSize[i]);

      if(allocation[i] != -1)
         printf("%d\n", allocation[i] + 1);
      else
         printf("Not Allocated\n");
   }
}

void bestFit(int blockSize[], int m, int processSize[], int n)
{
```

```
    int allocation[n];

    for(int i = 0; i < n; i++)
        allocation[i] = -1;

    for(int i = 0; i < n; i++)
    {
        int bestIdx = -1;
        for(int j = 0; j < m; j++)
        {
            if(blockSize[j] >= processSize[i])
            {
                if(bestIdx == -1 || blockSize[j] < blockSize[bestIdx])
                {
                    bestIdx = j;
                }
            }
        }

        if(bestIdx != -1)
        {
            allocation[i] = bestIdx;
            blockSize[bestIdx] = 0;
        }
    }

    printf("\n--- Best Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");

    for(int i = 0; i < n; i++)
    {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);
        if(allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void worstFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];
    for(int i = 0; i < n; i++)
        allocation[i] = -1;

    for(int i = 0; i < n; i++)
    {
```

```
        int worstIdx = -1;
        for(int j = 0; j < m; j++)
        {
            if(blockSize[j] >= processSize[i])
            {
                if(worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
                {
                    worstIdx = j;
                }
            }
        }

        if(worstIdx != -1)
        {
            allocation[i] = worstIdx;
            blockSize[worstIdx] = 0;
        }
    }

    printf("\n--- Worst Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");

    for(int i = 0; i < n; i++)
    {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);

        if(allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

int main()
{
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    int blockSize[m], block1[m], block2[m], block3[m];

    printf("Enter sizes of %d memory blocks:\n", m);

    for(int i = 0; i < m; i++)
    {
        scanf("%d", &blockSize[i]);
```

```
        block1[i] = blockSize[i];
        block2[i] = blockSize[i];
        block3[i] = blockSize[i];
    }

    printf("\nEnter number of processes: ");
    scanf("%d", &n);

    int processSize[n];

    printf("Enter sizes of %d processes:\n", n);

    for(int i = 0; i < n; i++)
    {
        scanf("%d", &processSize[i]);
    }

    firstFit(block1, m, processSize, n);
    bestFit(block2, m, processSize, n);
    worstFit(block3, m, processSize, n);

    return 0;
}
```

Result:



Program 7)

Question: Write a C program to simulate page replacement algorithms
a) FIFO b) LRU c) Optimal

Code:

```
#include <stdio.h>

#define MAX 100

int isPresent(int frames[], int num_frames, int page)
{
   for (int i = 0; i < num_frames; i++)
   {
      if (frames[i] == page)
         return 1;
   }

   return 0;
}


void printFrames(int frames[], int num_frames)
{
   printf("[");
   for (int i = 0; i < num_frames; i++)
   {
      if (frames[i] != -1)
         printf("%d ", frames[i]);
   }

   printf("]\n");
}


void FIFO(int pages[], int n, int num_frames)
{
   int frames[MAX];
   int page_faults = 0;
   int next = 0;

   for (int i = 0; i < num_frames; i++)
      frames[i] = -1;

   printf("\n--- FIFO Page Replacement ---\n");
```

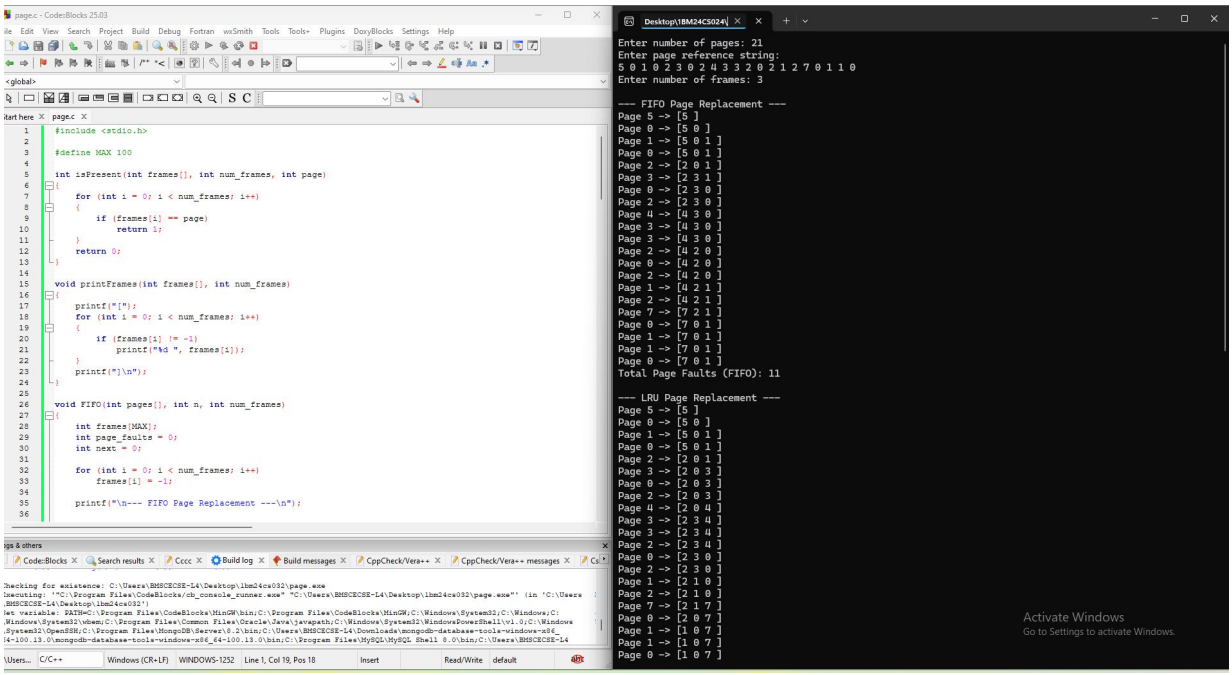
```
 for (int i = 0; i < n; i++)
    {
        int page = pages[i];

        if (!isPresent(frames, num_frames, page))
        {
            frames[next] = page;
            next = (next + 1) % num_frames;
            page_faults++;
        }

        printf("Page %d -> ", page);
        printFrames(frames, num_frames);
    }

    printf("Total Page Faults (FIFO): %d\n", page_faults);
}


void LRU(int pages[], int n, int num_frames)
{
    int frames[MAX];
    int lastUsed[MAX];
    int page_faults = 0;

    for (int i = 0; i < num_frames; i++)
    {
        frames[i] = -1;
        lastUsed[i] = -1;
    }

    printf("\n--- LRU Page Replacement ---\n");

    for (int i = 0; i < n; i++)
    {
        int page = pages[i];
        int found = 0;

        for (int j = 0; j < num_frames; j++)
        {
            if (frames[j] == page)
            {
                found = 1;
                lastUsed[j] = i;
                break;
```

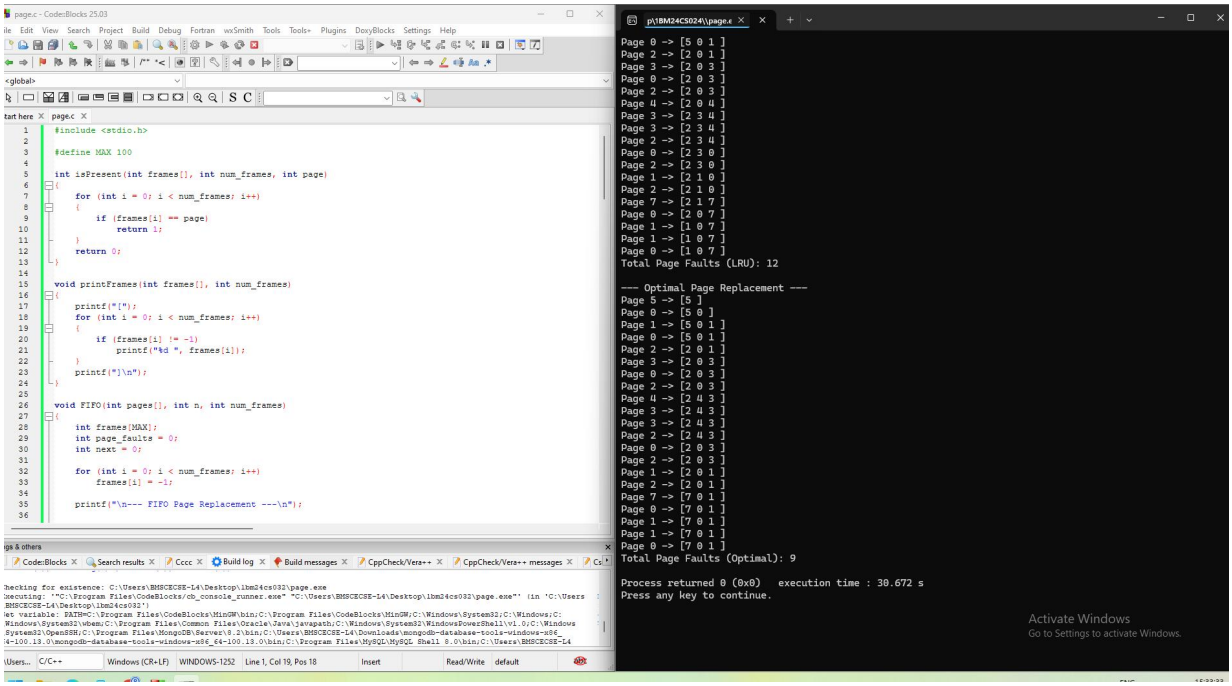
```
            }
        }

        if (!found)
        {

            int pos = -1;

            for (int j = 0; j < num_frames; j++)
            {
                if (frames[j] == -1)
                {
                    pos = j;
                    break;
                }
            }

            if (pos == -1)
            {
                int lruIndex = 0;

                for (int j = 1; j < num_frames; j++)
                {
                    if (lastUsed[j] < lastUsed[lruIndex])
                        lruIndex = j;
                }

                pos = lruIndex;
            }

            frames[pos] = page;
            lastUsed[pos] = i;
            page_faults++;
        }

        printf("Page %d -> ", page);
        printFrames(frames, num_frames);
    }

    printf("Total Page Faults (LRU): %d\n", page_faults);
}


void Optimal(int pages[], int n, int num_frames)
{
    int frames[MAX];
    int page_faults = 0;
```

```
// Initialize frames
for (int i = 0; i < num_frames; i++)
   frames[i] = -1;


printf("\n--- Optimal Page Replacement ---\n");

for (int i = 0; i < n; i++)
{

   int page = pages[i];

   if (!isPresent(frames, num_frames, page))
   {
      int pos = -1;

      for (int j = 0; j < num_frames; j++)
      {

         if (frames[j] == -1)
         {
            pos = j;
            break;
         }
      }

      if (pos == -1)
      {

         int farthest = i;
         int replaceIndex = -1;

         for (int j = 0; j < num_frames; j++)
         {
            int k;

            for (k = i + 1; k < n; k++)
            {
               if (frames[j] == pages[k])
                  break;
            }

            if (k == n)
            {
               replaceIndex = j;
               break;
```

```
                }

                if (k > farthest)
                {
                    farthest = k;
                    replaceIndex = j;
                }
            }

            pos = replaceIndex;
        }

        frames[pos] = page;
        page_faults++;
    }

    printf("Page %d -> ", page);
    printFrames(frames, num_frames);
  }

  printf("Total Page Faults (Optimal): %d\n", page_faults);
}


int main()
{
  int pages[MAX], n, num_frames;

  printf("Enter number of pages: ");
  scanf("%d", &n);

  printf("Enter page reference string:\n");
  for (int i = 0; i < n; i++)
  {
    scanf("%d", &pages[i]);
  }

  printf("Enter number of frames: ");
  scanf("%d", &num_frames);

  FIFO(pages, n, num_frames);
  LRU(pages, n, num_frames);
  Optimal(pages, n, num_frames);

  return 0;
}
```

Result:


Enter number of pages: 21
Enter page reference string:
5 0 1 0 2 3 0 2 4 3 3 2 0 2 1 2 7 0 1 1 0
Enter number of frames: 3
--- FIFO Page Replacement ---
Page 5 -> [5]
Page 0 -> [5 0]
Page 1 -> [5 0 1]
Page 0 -> [5 0 1]
Page 2 -> [2 0 1]
Page 3 -> [2 3 1]
Page 0 -> [2 3 0]
Page 2 -> [2 3 0]
Page 4 -> [4 3 0]
Page 3 -> [4 3 0]
Page 3 -> [4 3 0]
Page 2 -> [4 2 0]
Page 0 -> [4 2 0]
Page 2 -> [4 2 0]
Page 1 -> [4 2 1]
Page 2 -> [4 2 1]
Page 7 -> [7 2 1]
Page 0 -> [7 0 1]
Page 1 -> [7 0 1]
Page 1 -> [7 0 1]
Page 0 -> [7 0 1]
Total Page Faults (FIFO): 11
--- LRU Page Replacement ---
Page 5 -> [5]
Page 0 -> [5 0]
Page 1 -> [5 0 1]
Page 0 -> [5 0 1]
Page 2 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 2 -> [2 0 3]
Page 4 -> [2 0 4]
Page 3 -> [2 3 4]
Page 3 -> [2 3 4]
Page 2 -> [2 3 4]
Page 0 -> [2 3 0]
Page 2 -> [2 3 0]
Page 1 -> [2 1 0]
Page 2 -> [2 1 0]
Page 7 -> [2 1 7]
Page 0 -> [2 0 7]
Page 1 -> [1 0 7]
Page 1 -> [1 0 7]
Page 0 -> [1 0 7]


Page 0 -> [5 0 1]
Page 2 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 2 -> [2 0 3]
Page 4 -> [2 0 4]
Page 3 -> [2 3 4]
Page 3 -> [2 3 4]
Page 2 -> [2 3 4]
Page 0 -> [2 3 0]
Page 2 -> [2 3 0]
Page 1 -> [2 1 0]
Page 2 -> [2 1 0]
Page 7 -> [2 1 7]
Page 0 -> [2 0 7]
Page 1 -> [1 0 7]
Page 1 -> [1 0 7]
Page 0 -> [1 0 7]
Total Page Faults (LRU): 12
--- Optimal Page Replacement ---
Page 5 -> [5]
Page 0 -> [5 0]
Page 1 -> [5 0 1]
Page 0 -> [5 0 1]
Page 2 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 2 -> [2 0 3]
Page 4 -> [2 4 3]
Page 3 -> [2 4 3]
Page 3 -> [2 4 3]
Page 2 -> [2 4 3]
Page 0 -> [2 0 3]
Page 2 -> [2 0 3]
Page 1 -> [2 0 1]
Page 2 -> [2 0 1]
Page 7 -> [7 0 1]
Page 0 -> [7 0 1]
Page 1 -> [7 0 1]
Page 1 -> [7 0 1]
Page 0 -> [7 0 1]
Total Page Faults (Optimal): 9
Process returned 0 (0x0) execution time : 30.672 s
Press any key to continue.